

Cuando no usar tuberías en R

En R, las tuberías `%>%`, provenientes del paquete `magrittr`, permiten escribir código de forma más clara cuando tenemos una secuencia de operaciones. Son especialmente útiles cuando queremos transformar un objeto paso a paso.

Por ejemplo:

```
library(magrittr)
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
mtcars %>%
  select(mpg, cyl, hp) %>%
  filter(hp > 100) %>%
  arrange(desc(mpg))
```

```
##           mpg  cyl  hp
## Lotus Europa    30.4   4 113
## Hornet 4 Drive  21.4   6 110
## Volvo 142E      21.4   4 109
## Mazda RX4       21.0   6 110
## Mazda RX4 Wag   21.0   6 110
## Ferrari Dino    19.7   6 175
## Merc 280        19.2   6 123
## Pontiac Firebird 19.2   8 175
## Hornet Sportabout 18.7   8 175
## Valiant         18.1   6 105
## Merc 280C       17.8   6 123
## Merc 450SL      17.3   8 180
## Merc 450SE      16.4   8 180
## Ford Pantera L  15.8   8 264
## Dodge Challenger 15.5   8 150
## Merc 450SLC     15.2   8 180
## AMC Javelin     15.2   8 150
## Maserati Bora   15.0   8 335
## Chrysler Imperial 14.7   8 230
## Duster 360     14.3   8 245
## Camaro Z28     13.3   8 245
## Cadillac Fleetwood 10.4   8 205
## Lincoln Continental 10.4   8 215
```

Este código se lee casi como una oración:

Toma la base de datos `mtcars`, selecciona algunas columnas, filtra los autos con más de 100 caballos de fuerza y ordénalos de mayor a menor según `mpg`.

Sin embargo, las tuberías no siempre son la mejor opción. Aunque son muy útiles, también pueden hacer que el código sea difícil de leer, depurar o mantener si se usan de forma incorrecta.

1. No usar tuberías cuando la cadena es demasiado larga

Las tuberías funcionan muy bien cuando tenemos una secuencia corta y lineal de operaciones. Sin embargo, cuando una tubería tiene demasiados pasos, por ejemplo más de 10, el código puede volverse difícil de leer.

Ejemplo poco recomendable

```
mtcars %>%
  select(mpg, cyl, hp, wt, gear, carb) %>%
  filter(hp > 100) %>%
  mutate(rendimiento = mpg / hp) %>%
  mutate(peso_potencia = wt / hp) %>%
  group_by(cyl) %>%
  summarise(
    promedio_mpg = mean(mpg),
    promedio_hp = mean(hp),
    promedio_rendimiento = mean(rendimiento)
  ) %>%
  arrange(desc(promedio_rendimiento)) %>%
  filter(promedio_hp > 120)
```

```
## # A tibble: 2 x 4
##   cyl promedio_mpg promedio_hp promedio_rendimiento
##   <dbl>         <dbl>         <dbl>         <dbl>
## 1     6          19.7          122.          0.166
## 2     8          15.1          209.          0.0767
```

El código funciona, pero empieza a ser difícil de seguir. Si hay un error, puede ser complicado saber en qué paso ocurrió.

Mejor alternativa

Es preferible crear objetos intermedios con nombres significativos.

```
autos_seleccionados <- mtcars %>%
  select(mpg, cyl, hp, wt, gear, carb)

autos_filtrados <- autos_seleccionados %>%
  filter(hp > 100)

autos_con_variables <- autos_filtrados %>%
```

```

mutate(
  rendimiento = mpg / hp,
  peso_potencia = wt / hp
)

resumen_por_cilindros <- autos_con_variables %>%
  group_by(cyl) %>%
  summarise(
    promedio_mpg = mean(mpg),
    promedio_hp = mean(hp),
    promedio_rendimiento = mean(rendimiento)
  )

resultado_final <- resumen_por_cilindros %>%
  arrange(desc(promedio_rendimiento)) %>%
  filter(promedio_hp > 120)

resultado_final

```

```

## # A tibble: 2 x 4
##   cyl promedio_mpg promedio_hp promedio_rendimiento
##   <dbl>         <dbl>         <dbl>         <dbl>
## 1     6          19.7          122.           0.166
## 2     8          15.1          209.           0.0767

```

Esta versión es más larga, pero más clara. Además, permite revisar cada objeto intermedio si algo sale mal.

2. No usar tuberías cuando hay múltiples entradas importantes

Las tuberías funcionan mejor cuando hay un objeto principal que se va transformando. Por ejemplo, una base de datos que se filtra, se ordena y se resume.

Pero si tenemos dos o más objetos igual de importantes, la tubería puede ser confusa.

Ejemplo

Supongamos que tenemos dos tablas:

```

estudiantes <- data.frame(
  id = c(1, 2, 3),
  nombre = c("Ana", "Luis", "Marta")
)

notas <- data.frame(
  id = c(1, 2, 3),
  nota = c(8, 9, 7)
)

```

Podríamos escribir:

```
estudiantes %>%  
  left_join(notas, by = "id")
```

```
##   id nombre nota  
## 1  1    Ana    8  
## 2  2    Luis    9  
## 3  3    Marta    7
```

Este ejemplo todavía es aceptable, porque `estudiantes` es claramente la tabla principal.

Pero si las dos tablas tienen la misma importancia, puede ser más claro escribir:

```
tabla_final <- left_join(estudiantes, notas, by = "id")
```

```
tabla_final
```

```
##   id nombre nota  
## 1  1    Ana    8  
## 2  2    Luis    9  
## 3  3    Marta    7
```

En este caso, se ve explícitamente que estamos combinando dos tablas.

3. No usar tuberías cuando el proceso no es lineal

Las tuberías son lineales. Esto significa que el resultado de un paso pasa directamente al siguiente.

La estructura general es:

```
objeto %>%  
  paso_1() %>%  
  paso_2() %>%  
  paso_3()
```

Esto es útil cuando el proceso tiene una sola dirección.

Sin embargo, algunos problemas no son lineales. A veces necesitamos crear varias ramas, combinar resultados o usar varios objetos intermedios.

Ejemplo

Supongamos que queremos:

1. Filtrar autos con alta potencia.
2. Filtrar autos con bajo consumo.
3. Comparar ambos grupos.
4. Calcular resúmenes separados.

Esto no es una sola cadena lineal. Es mejor separarlo.

```

autos_alta_potencia <- mtcars %>%
  filter(hp > 150)

autos_bajo_consumo <- mtcars %>%
  filter(mpg < 20)

resumen_potencia <- autos_alta_potencia %>%
  summarise(
    promedio_hp = mean(hp),
    promedio_mpg = mean(mpg)
  )

resumen_consumo <- autos_bajo_consumo %>%
  summarise(
    promedio_hp = mean(hp),
    promedio_mpg = mean(mpg)
  )

resumen_potencia

```

```

##  promedio_hp promedio_mpg
## 1      215.6923      15.41538

```

```
resumen_consumo
```

```

##  promedio_hp promedio_mpg
## 1      191.9444      15.9

```

Aquí sería confuso intentar escribir todo en una sola tubería, porque realmente tenemos varios caminos de análisis.

4. No usar tuberías si dificultan la depuración

Depurar significa encontrar y corregir errores en el código.

Cuando usamos una tubería muy larga, si ocurre un error, puede ser difícil identificar exactamente en qué paso está el problema.

Ejemplo

```

mtcars %>%
  select(mpg, cyl, hp) %>%
  filter(potencia > 100) %>%
  arrange(desc(mpg))

```

Este código genera error porque la columna **potencia** no existe.

Una forma más fácil de depurar es separar el código:

```

datos_1 <- mtcars %>%
  select(mpg, cyl, hp)

datos_2 <- datos_1 %>%
  filter(hp > 100)

datos_3 <- datos_2 %>%
  arrange(desc(mpg))

datos_3

```

```

##           mpg  cyl  hp
## Lotus Europa    30.4   4 113
## Hornet 4 Drive  21.4   6 110
## Volvo 142E      21.4   4 109
## Mazda RX4       21.0   6 110
## Mazda RX4 Wag   21.0   6 110
## Ferrari Dino    19.7   6 175
## Merc 280         19.2   6 123
## Pontiac Firebird 19.2   8 175
## Hornet Sportabout 18.7   8 175
## Valiant          18.1   6 105
## Merc 280C        17.8   6 123
## Merc 450SL       17.3   8 180
## Merc 450SE       16.4   8 180
## Ford Pantera L   15.8   8 264
## Dodge Challenger 15.5   8 150
## Merc 450SLC      15.2   8 180
## AMC Javelin      15.2   8 150
## Maserati Bora    15.0   8 335
## Chrysler Imperial 14.7   8 230
## Duster 360       14.3   8 245
## Camaro Z28       13.3   8 245
## Cadillac Fleetwood 10.4   8 205
## Lincoln Continental 10.4   8 215

```

Ahora podemos revisar `datos_1`, `datos_2` y `datos_3` por separado.

5. No usar tuberías cuando el código pierde claridad

El objetivo de usar tuberías es escribir código más claro. Si una tubería hace que el código sea más difícil de entender, entonces no conviene usarla.

Por ejemplo, esta tubería puede ser confusa para alguien que está empezando:

```

mtcars %>%
  split(.$cyl) %>%
  lapply(function(x) {
    mean(x$mpg)
  })

```

```
## $'4'  
## [1] 26.66364  
##  
## $'6'  
## [1] 19.74286  
##  
## $'8'  
## [1] 15.1
```

Aunque el código es correcto, tal vez sea más claro escribirlo paso a paso:

```
autos_por_cilindros <- split(mtcars, mtcars$cyl)  
  
promedios_mpg <- lapply(autos_por_cilindros, function(x) {  
  mean(x$mpg)  
})  
  
promedios_mpg
```

```
## $'4'  
## [1] 26.66364  
##  
## $'6'  
## [1] 19.74286  
##  
## $'8'  
## [1] 15.1
```

La segunda versión ayuda a entender qué está ocurriendo.

Resumen

No conviene usar tuberías cuando:

1. La cadena de operaciones es demasiado larga.
2. Hay varios objetos de entrada igual de importantes.
3. El proceso no es lineal.
4. Se dificulta la depuración.
5. El código se vuelve menos claro.

La regla principal es:

Usa tuberías cuando ayuden a leer mejor el código.
No las uses cuando hagan que el código sea más difícil de entender.

Ejercicios

Ejercicio 1

El siguiente código usa una tubería larga. Reescríbelo usando objetos intermedios con nombres significativos.

```
mtcars %>%
  select(mpg, cyl, hp, wt) %>%
  filter(hp > 100) %>%
  mutate(rendimiento = mpg / hp) %>%
  group_by(cyl) %>%
  summarise(promedio = mean(rendimiento)) %>%
  arrange(desc(promedio))
```

```
## # A tibble: 3 x 2
##   cyl promedio
##   <dbl>   <dbl>
## 1     4   0.233
## 2     6   0.166
## 3     8   0.0767
```

```
select_columnas <- mtcars %>%
  select(mpg, cyl, hp, wt)

filter_hp_100 <- select_columnas %>%
  filter(hp>100)

crear_columna_rendimiento <- filter_hp_100 %>%
  mutate(rendimiento = mpg / hp)

agrupando_cyl <- crear_columna_rendimiento %>%
  group_by(cyl)

promedio_rendimiento <- agrupando_cyl %>%
  summarise(promedio=mean(rendimiento))

ordenado_promedio <- promedio_rendimiento %>%
  arrange(desc(promedio))
```

Ejercicio 2

Explica por qué la siguiente tubería puede ser difícil de depurar.

```
mtcars %>%
  select(mpg, cyl, hp) %>%
  filter(potencia > 100) %>%
  arrange(desc(mpg))
```

R// Debido que al correrlo lo lee como una función en cadena por lo que no nos especifica la línea donde está el error por lo que toca buscar el error a ojo y eso en un código largo puede ser difícil y tedioso

Ejercicio 3

Corrige el siguiente código y reescríbelo de forma que sea más fácil revisar cada paso.

#el error estaba en millas esa columna no existe era mpg

```
mtcars %>%
  select(mpg, cyl, hp) %>%
  mutate(relacion = millas / hp) %>%
  arrange(desc(relacion))

seleccion_variable <- mtcars %>%
  select(mpg, cyl, hp)

creacion_relacion <- seleccion_variable %>%
  mutate(relacion=mpg/hp)

ordenamiento_relacion <- creacion_relacion %>%
  arrange(desc(relacion))
```

Ejercicio 4

Decide si conviene o no usar tuberías en el siguiente caso. Justifica tu respuesta.

Queremos:

1. Tomar iris.
2. Seleccionar Sepal.Length, Sepal.Width y Species.
3. Filtrar las flores de la especie setosa.
4. Ordenar de mayor a menor según Sepal.Length.

R// si conviene usar ya que lo que se hace es lineal (seleccionar,filtrar,ordenar) se veria ordenado he intuitivo usando tuberias

Ejercicio 5

Decide si conviene o no usar una sola tubería para este análisis.

Queremos:

1. Crear un grupo de autos con `hp > 150`.
2. Crear otro grupo de autos con `mpg > 25`.
3. Calcular un resumen para cada grupo.
4. Comparar los resultados.

R// En este caso no, seria mas ordenado y correcto hacer tuberias individuales para cada operacion

ya que se crean grupos demasiados distintos que podrian usarse por separado para la comparacion de resultados es conveniente tenerlos separados

Ejercicio 6

El siguiente código tiene muchas operaciones. Reescríbelo usando objetos intermedios.

```
iris %>%
  select(Sepal.Length, Sepal.Width, Petal.Length, Petal.Width, Species) %>%
  filter(Sepal.Length > 5) %>%
  mutate(
    razon_sepalo = Sepal.Length / Sepal.Width,
    razon_petalo = Petal.Length / Petal.Width
  ) %>%
  group_by(Species) %>%
  summarise(
    promedio_sepalo = mean(razon_sepalo),
    promedio_petalo = mean(razon_petalo)
  ) %>%
  arrange(desc(promedio_petalo))
```

```
## # A tibble: 3 x 3
##   Species      promedio_sepalo promedio_petalo
##   <fct>          <dbl>          <dbl>
## 1 setosa          1.44            6.24
## 2 versicolor      2.16            3.23
## 3 virginica       2.24            2.78
```

```
seleccion <- iris %>%
  select(Sepal.Length, Sepal.Width, Petal.Length, Petal.Width, Species)

filtrado <- seleccion %>%
  filter(Sepal.Length > 5)

creacion_razon <- filtrado %>%
  mutate(
    razon_sepalo = Sepal.Length / Sepal.Width,
    razon_petalo = Petal.Length / Petal.Width)

agrupado_species <- creacion_razon %>%
  group_by(Species)

promedios_sepalo_petalo <- agrupado_species %>%
  summarise(
    promedio_sepalo = mean(razon_sepalo),
    promedio_petalo = mean(razon_petalo))
```

```
ordenamiento_promedio_petalos <- promedios_sepalo_petalos %>%  
  arrange(desc(promedio_petalos))
```